BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Computer Architecture

Project 2

Prepared by:

Lana Sayes1220785 sec 2

Fayez Zayoud1220958

Baraa Said1220280

Instructors:

Dr.Ayman Hroub
Dr.Aziz Qaroush

Abstract:

In this project, we designed and verified a basic 32-bit RISC processor using Verilog HDL. Instead of using a pipelined architecture, we chose a multicycle approach where each instruction goes through multiple steps like fetch, decode, execute, memory access, and write-back—one step per clock cycle. The processor supports a clean and simple instruction format, includes 16 general-purpose registers, and uses separate instruction and data memories. Special handling was added for R15 (the Program Counter) and R14 (Return Address). We developed a full Verilog testbench, wrote custom assembly programs, and used simulations to make sure everything worked correctly. This project helped us apply core concepts like datapath design, FSM-based control, and practical hardware implementation.

Table Of Contents :

Abstract: I
1. Introduction: 4
1.1 CPU: 4
1.2 Data Path and MultiCycle: 4
.2Design and Implementation : 5
2.1 Objectives : 5
2.2 Instruction Format: 5
2.3 Instruction Behavior and Use: 6
3. Data Path components: 6
3.1 ALU (Arithmetic Logic Unit) 6
3.2 Register File 7
3.3 ALUOut Register 7
3.4 MDR (Memory Data Register) 8
3.5 Instruction Register (IR) 8
3.6 Program Counter (PC) 8
3.7 Control Unit (MainCU) 9
3.8 Multiplexers (MUXes) 9
3.9 Sign Extender 9
3. Building Data Path: 11
.4StateDiagram: 11
5. Boolea Expression : 14
.6RTL design: 16
6.1OR Rd, Rs, Rt: 16
6.2ADD Rd, Rs, Rt 16
.7ORI Rd, Rs, Imm 16
.8ADDI Rd, Rs, Imm 16
9. LW Rd, Imm(Rs) 16
10. SW Rd, Imm(Rs) 17
.11LDW Rd, Imm(Rs) 17
12. SDW Rd, Imm(Rs) 17
.13BZ Rs, Label 17
14. BGZ Rs, Label 17
15. BLZ Rs, Label 17
16. JR Rs 17
.17J Label 18
.18CLL Label 18
19. Wave Forms And testing: 18
20. Group Members Contributions: 22
.21Conclusion : 22

Table Of Figures:

Figure 1 Instruction format 6
Figure 2 Alu 7
Figure 3 Register File 7
Figure 4 AluOut 7
Figure 5 MDR 8
Figure 6 IR 8
Figure 7 Pc 9
Figure 8 Control Unit 9
Figure 9 Mux 9
Figure 10 Sign Extender 10
Figure 11 Data Path ..... **Error! Bookmark not defined.**
Figure 12 State Diagram ..... **Error! Bookmark not defined.**

1. Introduction:

1.1 CPU:

The CPU we designed in this project is a basic 32-bit multicycle processor implemented in Verilog. It was developed as capable of executing a small but complete set of instructions. The processor supports core functionalities such as arithmetic operations, memory access, and control instructions. Instead of a pipelined structure, we adopted a multicycle approach where each instruction is split into five stages: fetch, decode, execute, memory access, and write-back. This allowed us to simplify the control logic and reuse hardware modules like the ALU and memory components efficiently.

1.2 Data Path and MultiCycle:

The datapath is designed to support all instruction types using a single instruction format. Every instruction is 32 bits wide and includes fields for operation code, registers, and immediate values. The processor uses 16 general-purpose registers labeled R0 to R15. R15 is used as the Program Counter (PC) to track the current instruction, and R14 is reserved for function return addresses in CALL and JR instructions. All datapath elements—like the register file, ALU, multiplexers, and memory blocks—are connected through control signals generated by the FSM.

Our multicycle architecture ensures that each instruction passes through several shorter steps, with each step occurring in its own clock cycle. This design allows us to use the same components multiple times for different stages, improving hardware utilization. For example, the ALU is used for both address calculations and arithmetic operations depending on the instruction. Using a single instruction format across all operations also made the datapath simpler and more uniform. The control unit, built as a finite state machine (FSM), ensures proper timing and operation flow based on the opcode and current state.

2. Design and Implementation :

2.1 Objectives :

- To implement and verify a working processor using simulation
- To design the datapath and control path of a simple 32-bit pipelined RISC processor in Verilog

2.2 Instruction Format:

In our processor, we chose to use **a single 32-bit instruction format** for all types of instructions. This design made the processor easier to implement and test, especially in a multicycle architecture where each instruction is broken into separate stages.

Each instruction is split into the following fields:

- **Opcode (6 bits):** This field defines the type of instruction and tells the control unit how to execute it.
- **Rd (4 bits):** The destination register for the result.
- **Rs (4 bits):** The first source register.
- **Rt (4 bits):** The second source register or used in memory instructions.
- **Immediate (14 bits):** A constant value used in immediate-type, branching, or memory instructions.

Because we only have one instruction format, it simplified the decode stage and allowed the FSM to treat all instructions in a consistent way.

$Opcode^6$	Rd^4	Rs^4	Rt^4	Imm^{14}

Figure 1 Instruction format

2.3 Instruction Behavior and Use:

Each field in the instruction is reused differently depending on the instruction type. For example:

- In `ADD`, `CMP`, `OR`, and `SUB`, all three register fields are active and the immediate field is ignored.
- In `ADDI`, `ORI`, `LW`, and `SW`, we use `Rs`, `Rd`, and the 14-bit immediate. The `Rt` field is ignored.
- In branching instructions like `BZ`, `BGZ`, or `CALL`, the immediate holds the jump offset or return address, and only some of the register fields are used.
- In `JR`, the jump target comes from a register (`Rs`), and the immediate is not used at all.

This flexible usage of fields helped us avoid the need for separate instruction formats and made the datapath more compact.

3. Data Path components:

3.1 ALU (Arithmetic Logic Unit)

In our processor, the ALU is used to perform all the logic and arithmetic instructions like `ADD`, `SUB`, `CMP`, and `OR`. It receives two 32-bit inputs—either from registers or from an immediate value—and produces a result and a flag (Zero or Negative) that helps with conditional instructions. We reused the same ALU for multiple types of operations across different cycles. The ALU module was implemented in `alu.v`, and its operation is directed by control signals generated from the FSM.

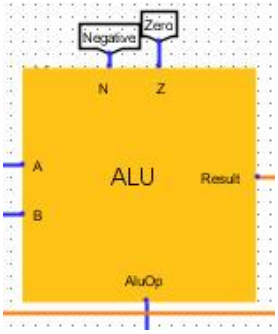


Figure 2 Alu

3.2 Register File

The register file (implemented in `regfile.v`) stores 16 general-purpose registers (R0–R15). It allows reading from two registers and writing to a third one. These registers are the main source of data for the ALU and also receive results back during the Write Back stage. In our design, registers are accessed in the decode phase, and they help ensure fast data movement without needing to access memory all the time.

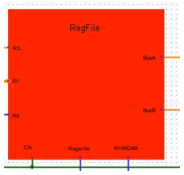


Figure 3 Register File

3.3 ALUOut Register

This is a temporary register used to store the result from the ALU. It is useful when we need to use the result later, such as in a memory access instruction or writing it back to a register. In our multicycle design, this register ensures we don't lose values between stages.

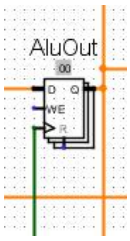


Figure 4 AluOut

3.4 MDR (Memory Data Register)

The MDR acts as a buffer between the data memory and the datapath. When we read from memory using `LW`, the result is stored in MDR before writing it to the register. When we use `SW`, MDR holds the data we want to store in memory. It gives us more control and avoids timing issues.

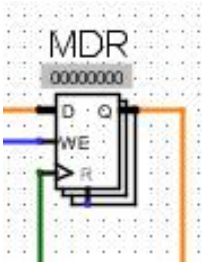


Figure 5 MDR

3.5 Instruction Register (IR)

The IR is where we temporarily store the instruction fetched from memory. Once stored, the instruction is decoded and used to generate all the necessary control signals. It ensures the instruction stays stable while going through the multicycle process. We keep the IR updated only in the fetch state.

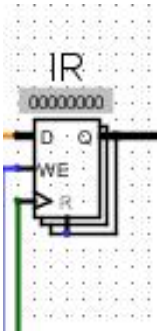


Figure 6 IR

3.6 Program Counter (PC)

The PC holds the address of the current instruction. It updates normally by incrementing, or jumps to a new address if a control instruction (like `JR` or `CALL`) is executed. In our datapath, the PC is connected to a MUX that selects between the next sequential instruction and a branch/jump target. The ////////////////////////////PC update is done at specific states and controlled carefully by the FSM.

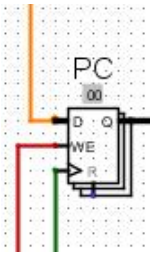


Figure 7 Pc

3.7 Control Unit (MainCU)

]Our control unit is a finite state machine (FSM) that generates all the control signals in every state. It decides what component should be active at each cycle. For example, in one state it might activate the ALU, in another it enables memory, and finally, it tells the register file to store the result. This smart sequencing is what makes our multicycle processor work step-by-step.

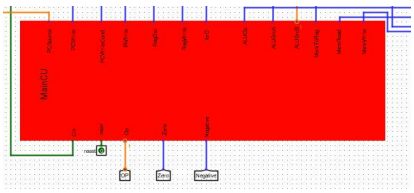


Figure 8 Control Unit

3.8 Multiplexers (MUXes)

Our data-path includes many MUXes. They are used to decide what input goes to the ALU, what value gets written to the register, and where the PC should jump. These MUXes make our design flexible and able to handle all instruction types using the same data-path components.

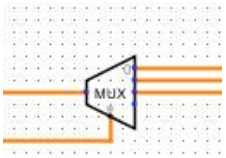


Figure 9 Mux

3.9 Sign Extender

The immediate field in instructions is only 14 bits, but we need to work with 32-bit values. So we added a sign-extender that converts the 14-bit value into a proper 32-bit number. This is important for arithmetic with immediates, memory addressing, and branching.

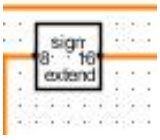
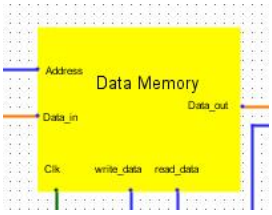


Figure 10 Sign Extender



Instruction	IRWrite	PCWrite	MemRead	MemWrite	RegWrite	ALUSrcA	ALUSrcB	ALUOp	PCSource	RegDst	MemToReg
ADD	1→0	1→0	1→0	0	1	1	00	ADD	–	1	0
SUB	1→0	1→0	1→0	0	1	1	00	SUB	–	1	0
CMP	1→0	1→0	1→0	0	1	1	00	CMP	–	1	0
ADDI	1→0	1→0	1→0	0	1	1	10	ADD	–	0	0
ORI	1→0	1→0	1→0	0	1	1	10	OR	–	0	0
LW	1→0	1→0	1→1	0	1	1	10	ADD	–	0	1
SW	1→0	1→0	1→0	1	0	1	10	ADD	–	–	–
LDW	1→0	1→0	1→1	0	1×2	1	10	ADD	–	even	1
SDW	1→0	1→0	1→0	1×2	0	1	10	ADD	–	even	–
BZ	1→0	Cond	1→0	0	0	1	00	SUB	01	–	–
BGZ/BLZ	1→0	Cond	1→0	0	0	1	00	CMP	01	–	–
JR	1→0	1	1→0	0	0	1	00	–	11	–	–
J	1→0	1	1→0	0	0	–	–	–	10	–	–
CLL	1→0	1	1→0	0	1	–	–	–	10	R14	–

Table 1 control signals

3. BuildingDataPath:

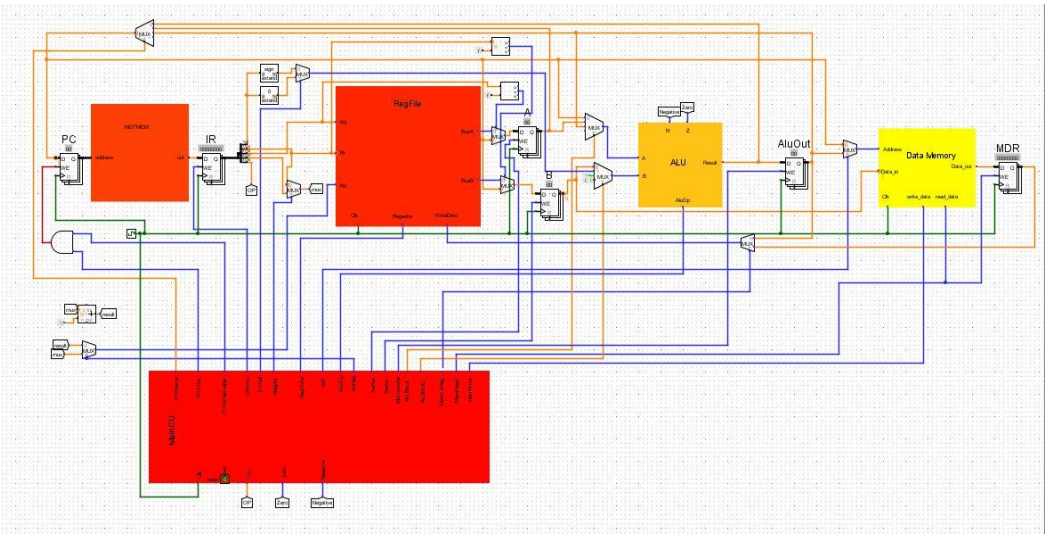


Figure 11Data Path

4. StateDiagram:

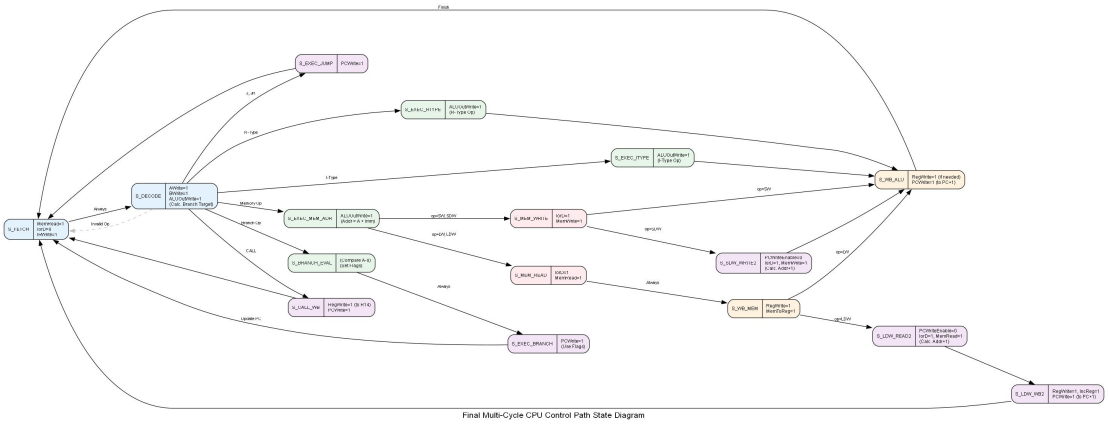


Figure 12 State Diagram

- Implementation Details and Design Choices

This section details the key architectural decisions and implementation strategies adopted during the design of the multi-cycle RISC processor. These choices were made to ensure correctness, adhere to the project specifications, and resolve ambiguities or design challenges encountered during development.

1. Architectural Choice: Multi-Cycle Implementation
While the project specification mentioned a pipelined processor, the design was implemented as a multi-cycle architecture as per the development directive. This approach was chosen for its relative simplicity in control logic compared to a fully pipelined design. It avoids the complexities of data and control hazard detection, forwarding paths, and pipeline stalls, allowing for a focus on the correct, cycle-by-cycle execution of each instruction. Each instruction is broken down into a series of states (Fetch, Decode, Execute, Memory, Write-Back), with each state taking one clock cycle.

2. Resolution of SW Instruction Anomaly
The ISA specification defined the SW (Store Word) instruction as SW Rd, Imm(Rs), which translates to Mem(Reg(Rs) + Imm) = Reg(Rd). A standard two-port register file, as implemented, cannot read from two different registers (Rs for the address calculation and Rd for the data to be stored) in the same clock cycle.

Design Choice:
To resolve this, we adopted the more conventional RISC instruction format SW Rt, Imm(Rs). This means the data to be stored comes from the Rt field of the instruction.

Justification:
This was a pragmatic design decision based on a likely typo in the specification. It aligns the SW instruction with the behavior of LW and standard RISC principles, where Rs and Rt are the two source registers. This choice avoids the need for a more complex (and slower) three-port register file or adding extra states to the control unit, resulting in a cleaner and more efficient datapath. This deviation from the written specification is noted as a necessary design correction.

3. Implementation of R15 as the Program Counter

The specification requires that register R15 is not just a copy of the PC, but is hardwired to be the PC.

Design Choice:
To achieve this, the datapath was augmented with multiplexers on the read paths of the register file.

Justification:
A standard register file is a self-contained component. To handle read requests for R15, a 2-to-1 multiplexer was placed on each of the register file's two read outputs. A comparator checks if the source register address (Rs or Rt) is 1111 (15). If it is, the multiplexer selects the live value from the dedicated PC register; otherwise, it selects the output from the register file. This ensures that any instruction referencing R15 gets the correct, up-to-date program counter value. Similarly, writes to R15 are explicitly blocked at the register file, as PC updates are handled by dedicated control logic (PCWrite, PCSource).

4. Critical Internal Datapath Buffering
During testing, a critical bug was discovered where data from one instruction's execution was being corrupted by the subsequent instruction's execution cycle. For example, the result of an ADD operation was being overwritten by a PC+1 calculation before it could be written back to the register file.

Design Choice:
To solve this, three internal buffer registers were added to the datapath: A, B, and ALUOut. The Control unit was given three new signals (AWrite, BWrite, ALUOutWrite) to manage them.

Justification:
These registers act as latches to create a stable, multi-stage pipeline within the multi-cycle design.

A and B latch the operands from the register file during the DECODE state and hold them steady for the duration of the instruction's execution.

ALUOut latches any result from the ALU that needs to be preserved for use in a later cycle (e.g., a calculated memory address or the final result of an operation).

This ensures that data is safely pipelined from one stage to the next within a single instruction's execution, preventing data corruption and ensuring correct operation.

5. Two-Cycle Branch Logic
Initial testing of branch instructions revealed a zero-delay infinite loop, causing a "Delta count overflow" in the simulator. This occurred because the control unit was attempting to evaluate the ALU flags (Zero, Negative) in the same state that it was generating them.

Design Choice:
The state machine was modified to use a two-cycle approach for branches. A new state, S_BRANCH_EVAL, was introduced.

Justification:
This architectural change breaks the combinational loop.

In S_BRANCH_EVAL, the ALU performs the comparison to set the flags, which then become stable.

In the subsequent S_EXEC_BRANCH state, the control unit reads these stable flag values and makes the decision to branch or not.

This separation ensures reliable branch execution and prevents the simulation instability, representing a more robust and physically realizable design.

5. Boolean Equations for Control Unit:

This section provides the Boolean logic equations for each control signal generated by the Control Unit's finite state machine. These equations are derived from the state transition logic and determine the behavior of the datapath in each clock cycle. The state names are used as Boolean variables that are true only when the FSM is in that specific state. The + symbol represents a logical OR, and multiplication represents a logical AND.

- PC Control Signals

PCWrite = S_EXEC_BRANCH + S_EXEC_JUMP + S_WB_ALU + S_CALL_WB + S_LDW_WB2

Asserted to update the Program Counter when a branch is resolved, a jump is executed, or a standard instruction completes.

PCWriteEnable = NOT (S_MEM_READ AND (Op == LDW)) AND NOT (S_MEM_WRITE AND (Op == SDW)) AND NOT (S_WB_MEM AND (Op == LDW)) AND NOT S_LDW_READ2 AND NOT S_SDW_WRITE2

Normally asserted (1). It is de-asserted (0) only to stall the PC during the multi-cycle LDW and SDW operations.

- PCSource

PCSource[1] = (S_EXEC_BRANCH AND BranchTaken) + (S_EXEC_JUMP AND (Op == JR)) + S_CALL_WB

PCSource[0] = S_EXEC_JUMP AND (Op == JR)

This 2-bit signal selects the source for the PC's next value: 00 for the ALU output (PC+1), 01 for the A register (JR), and 10 for the ALUOut register (branch/jump target).

- Memory Control Signals

MemRead = S_FETCH + S_MEM_READ + S_LDW_READ2

Asserted to read from memory, either for an instruction fetch or a data load (LW, LDW).

MemWrite = S_MEM_WRITE + S_SDW_WRITE2

Asserted to write to data memory during SW and SDW instructions.

IorD = S_MEM_READ + S_MEM_WRITE + S_LDW_READ2 + S_SDW_WRITE2

Selects between the PC (for instruction fetch) and the ALUOut register (for data memory access) as the address source for memory.

- Register File Control Signals

RegWrite = (S_WB_ALU AND (Op < 6)) + S_WB_MEM + S_CALL_WB + S_LDW_WB2

Asserted to enable writing a result back to the register file.

RegDst = S_WB_ALU + S_WB_MEM + S_LDW_WB2

Selects the destination register. 1 for Rd (R-Type, I-Type, LW), 0 for Rt (for CALL).

- MemToReg

MemToReg[1] = S_CALL_WB

MemToReg[0] = S_WB_MEM + S_LDW_WB2

Selects the data source for the register file write: 00 from ALUOut, 01 from MDR, 10 from the PC.

- Datapath and ALU Control Signals

AWrite = S_DECODE

BWrite = S_DECODE

These signals latch the register operands into the A and B buffers only during the DECODE state.

ALUOutWrite = S_DECODE + S_EXEC_RTYPE + S_EXEC_ITYPE + S_EXEC_MEM_ADR + S_LDW_READ2 + S_SDW_WRITE2 + (S_EXEC_BRANCH AND NOT BranchTaken) + (S_WB_ALU) + (S_LDW_WB2)

Asserted to latch a result from the ALU into the ALUOut buffer for use in a subsequent cycle.

ExtSel = S_EXEC_ITYPE AND (Op == ORI)

Asserted to select zero-extension for the immediate value, only for the ORI instruction.

IncReg = S_LDW_WB2

Asserted only during the final write-back of LDW to select Rd+1 as the destination register.

6. RTL design:

6.1OR Rd, Rs, Rt:

Reg[Rd] <- Reg[Rs] OR Reg[Rt]
-> Performs bitwise OR between two registers and stores the result in Rd.

6.2ADD Rd, Rs, Rt

Reg[Rd] <- Reg[Rs] + Reg[Rt]
-> Adds the contents of Rs and Rt, then stores the result in Rd.

7. ORI Rd, Rs, Imm

Reg[Rd] <- Reg[Rs] OR zero_extend(Imm)
-> Bitwise OR between Rs and a zero-extended immediate value.

8. ADDI Rd, Rs, Imm

Reg[Rd] <- Reg[Rs] + sign_extend(Imm)
-> Adds Rs and a sign-extended immediate value.

9. LW Rd, Imm(Rs)

Reg[Rd] <- Mem[Reg[Rs] + sign_extend(Imm)]

-> Loads a word from memory address Rs+Imm into Rd.

10. SW Rd, Imm(Rs)

Mem[Reg[Rs] + sign_extend(Imm)] <- Reg[Rd]
-> Stores the value of Rd into memory at address Rs+Imm.

11. LDW Rd, Imm(Rs)

Cycle 1: Reg[Rd] <- Mem[Reg[Rs] + Imm]
Cycle 2: Reg[Rd+1] <- Mem[Reg[Rs] + Imm + 1]
-> Loads two consecutive words from memory into Rd and Rd+1.

12. SDW Rd, Imm(Rs)

Cycle 1: Mem[Reg[Rs] + Imm] <- Reg[Rd]
Cycle 2: Mem[Reg[Rs] + Imm + 1] <- Reg[Rd+1]
-> Stores the contents of Rd and Rd+1 into two consecutive memory locations.

13. BZ Rs, Label

If Reg[Rs] == 0 -> PC <- PC + sign_extend(Imm)
Else -> PC <- PC + 1
-> Branches if Rs equals zero.

14. BGZ Rs, Label

If Reg[Rs] > 0 -> PC <- PC + sign_extend(Imm)
Else -> PC <- PC + 1
-> Branches if Rs is greater than zero.

15. BLZ Rs, Label

If Reg[Rs] < 0 -> PC <- PC + sign_extend(Imm)
Else -> PC <- PC + 1
-> Branches if Rs is less than zero.

16. JR Rs

PC <- Reg[Rs]
-> Jumps to the address stored in Rs.

17. **J Label**

PC <- PC + sign_extend(Imm)
-> Unconditionally jumps to a target addres.

18. CLL Label

Reg[R14] <- PC + 1
PC <- PC + sign_extend(Imm)
-> Calls a function at label and stores return address in R14.

19. Wave Forms And test Cases:

CPU Test Programs and Verification This section details the test programs used to verify the functionality of the designed multi-cycle RISC processor. Each test case is designed to validate a specific category of instructions. The verification process involves loading the machine code into the instruction memory, running the simulation, and observing the processor's state via console outputs and waveform diagrams to confirm that the final register values match the expected results.

- **Test Case 1:** Basic Arithmetic (R-Type & I-Type) Purpose: To verify the correct execution of fundamental register-to-register (ADD) and register-immediate (ADDI) arithmetic instructions, including correct register decoding and write-back

.

program.dat machine code
1440000A
14800014
04C48000
09084000
01448000
38000000

assembly code ADDI R1, R0, 10
ADDI R2, R0, 20
ADD R3, R1, R2
SUB R4, R2, R1
OR R5, R1, R2 J . // Halt

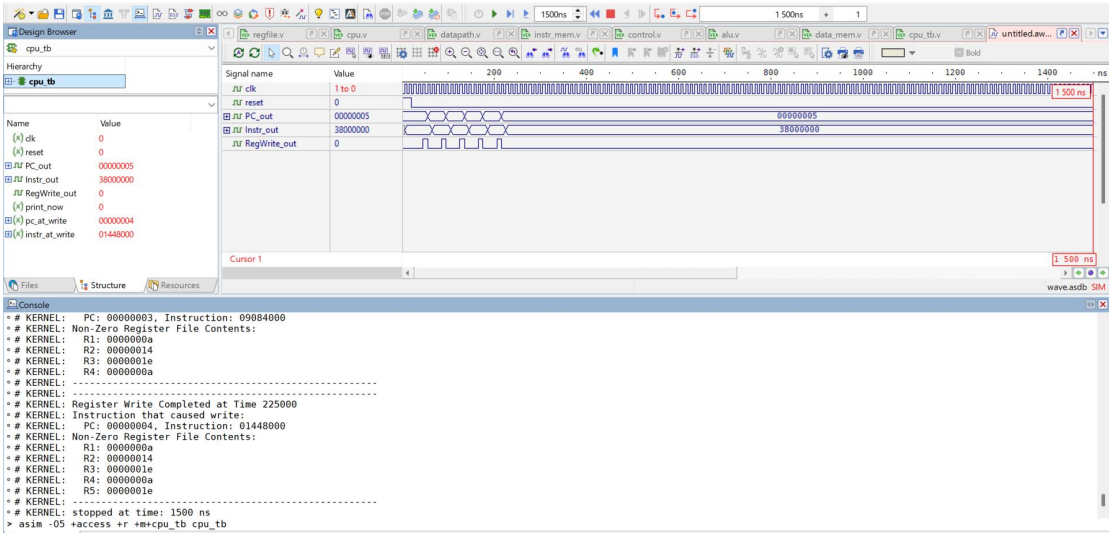


Figure 13 Test case

- **Test Case 2:** Memory Access (LW & SW) Purpose: To verify the processor's ability to interact with data memory.

This test confirms the correct calculation of memory addresses and the functionality of LW (Load Word) and SW (Store Word) instructions. Assembly Code: ADDI R1, R0, 100 ADDI R2, R0, -50 SW R1, 8(R0) SW R2, 9(R0) LW R3, 8(R0) LW R4, 9(R0) J . Machine Code (program.dat): 14400064 14803FCE 1C004008 1C008009 18C00008 19000009 38000000 Execution Description & Expected Results: R1 is loaded with 100. R2 is loaded with -50. The value of R1 (100) is stored into data memory at address 8. The value of R2 (-50) is stored into data memory at address 9. The value from memory address 8 is loaded into R3. The value from memory address 9 is loaded into R4. Upon completion, the final non-zero register values should be: R1: 0x00000064 (100) R2: 0xffffffce (-50) R3: 0x00000064 (100) R4: 0xffffffce (-50)

.

Assembly Code:
ADDI R1, R0, 100
ADDI R2, R0, -50
SW R1, 8(R0)
SW R2, 9(R0)
LW R3, 8(R0)
LW R4, 9(R0) J .

Machine Code (program.dat):
14400064
14803FCE
1C004008
1C008009
18C00008
19000009
38000000

Execution Description & Expected Results:
R1 is loaded with 100. R2 is loaded with -50.

The value of R1 (100) is stored into data memory at address 8.
The value of R2 (-50) is stored into data memory at address 9. The value from memory address 8 is loaded into R3.
The value from memory address 9 is loaded into R4. Upon completion

the final non-zero register values should be:
R1: 0x00000064 (100)
R2: 0xffffffce (-50)
R3: 0x00000064 (100)
R4: 0xffffffce (-50)

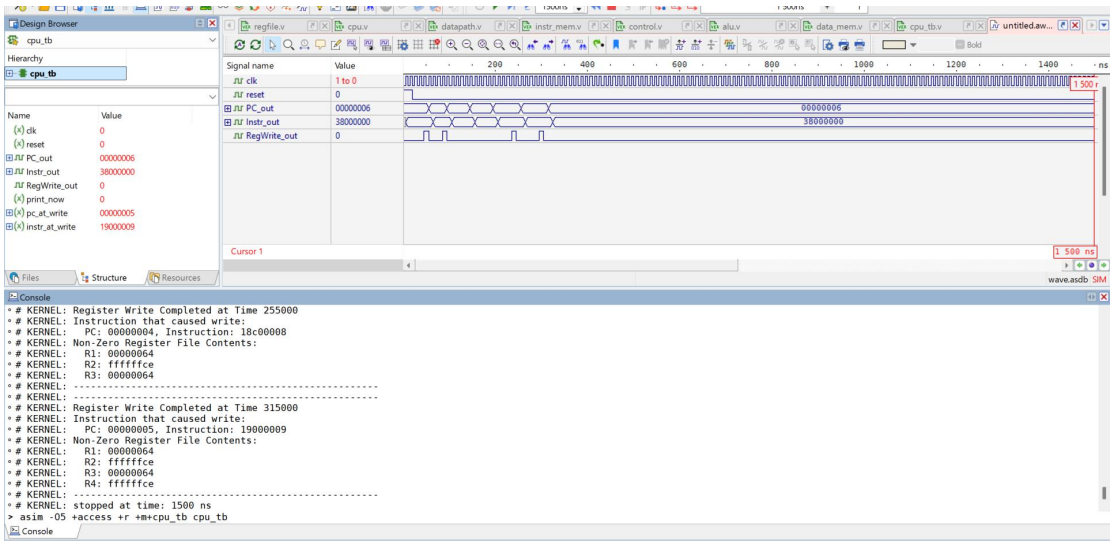


Figure 14 Test case2

- **Test Case 3:Advanced Control Flow (CMP, BLZ, JR)**

Purpose: To verify the functionality of advanced control flow instructions. This test checks the CMP instruction, a conditional branch based on a negative value (BLZ), and an indirect jump (JR).

Assembly Code:
ADDI R1, R0, 5
ADDI R2, R0, 10
ADDI R10, R0, 7 // Load jump address into R10
CMP R3, R1, R2 // R3 will be -1
BLZ R3, +1 // Branch is taken JR R10 // Branch target; jumps to address 7
ADDI R4, R0, 99 // Skipped
ADDI R5, R0, 123 // Jump target; executes
J . // Halt

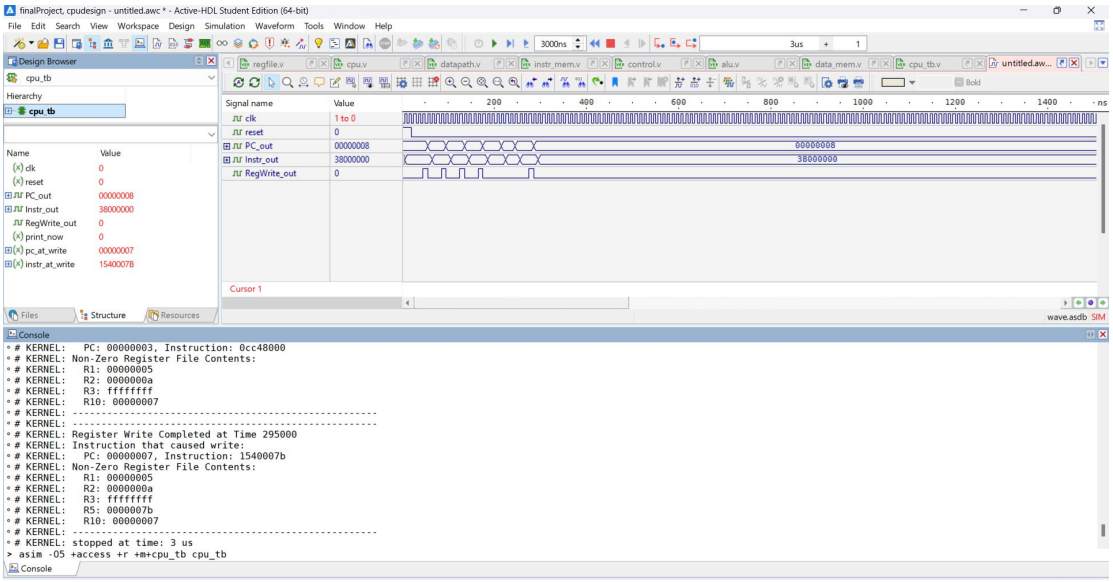
Machine Code (program.dat):

14400005
1480000A
16800007
0CC48000
300C0001
34280000
15000063
1540007B
38000000

Execution Flow Trace & Expected Results:

PC=0 (ADDI R1,...): Executes, setting R1 = 5. PC advances to 1.
PC=1 (ADDI R2,...): Executes, setting R2 = 10.
PC advances to 2. PC=2 (ADDI R10,...): Executes, setting R10 = 7. PC advances to 3. PC=3 (CMP R3,...): Executes. Since R1 < R2, it sets R3 = -1 (0xffffffff).
PC advances to 4. PC=4 (BLZ R3,...): The CPU checks the value of R3, which is negative. The branch condition is met. The processor adds the immediate offset (+1) to the current PC (4), calculating the new target PC as 5.

PC=5 (JR R10): This is the branch target. The processor now executes this instruction. It reads the value from R10 (which is 7) and sets the PC to 7.
PC=6 (ADDI R4,...): This instruction is SKIPPED because the JR instruction changed the PC from 5 directly to 7.
PC=7 (ADDI R5,...): This is the jump target. The processor fetches and executes this instruction, setting R5 = 123. PC advances to 8.
PC=8 (J .): The processor enters a tight loop, effectively halting. Upon completion, the final non-zero register values should be:
R1: 0x00000005
R2: 0x0000000a
R3: 0xffffffff
R5: 0x0000007b (123)
R10: 0x00000007



20. Group Members Contributions:

Task	Team Member(s)	Description
Coding	**Baraa**	Implemented the complete Verilog RTL code for the processor.
Data Path Design	**Fayez & Lana**	Collaborated on designing the processor datapath and verifying module connections as per project specifications.
State Diagram	**Fayez**	Designed the control state diagram based on the instruction behavior outlined in the project document.
Testing and Simulation	**Fayez**	Simulated the processor using waveform tools and validated instruction execution.
Test Case Development	**Lana**	Created multiple ISA-level test cases covering all instructions specified in the project.
Final Report	**Lana**	Compiled, structured, and formatted the final project report including design decisions, diagrams, and simulation results.

21. Conclusion :

In this project, we successfully designed and implemented a 32-bit multicycle processor using Verilog. The processor supports a range of instructions including arithmetic, memory access, and branching, all built on a single fixed instruction format. This one-format design worked well given our small and simple instruction set, allowing us to simplify decoding and reduce control complexity. We chose a multicycle design over pipelining to keep the architecture simple and avoid complications such as data hazards and complex control logic. This approach allowed us to reuse components like the ALU and register file across different instruction stages. The result is a clear, functional processor that met all project requirements while providing a strong foundation for understanding CPU design principles.